

BPMS, APIs & Cloud-native

Von der Prozesslandkarte zur integrierten Ausführung –
E2E-Integration als Steuerungsarchitektur.

Lehrskript – Tag 6 von 16

Lernpfad:
Wissen → Anwendung → Management

Dozent:
Can Siebert

Bearbeitungsdauer:
1 Kurstag

Format:
Theorie · Fallstudie · Coaching

Lernziele dieses Tages

Bisher haben wir Modelle entworfen, in einem Repository steuerbar gemacht und über Capabilities/MDA in eine Architekturlogik gebracht. Heute treten wir in die Schicht ein, in der Prozesse tatsächlich *laufen*: BPMS-Plattformen führen Workflows aus, APIs verbinden Systeme als Verträge, Cloud-native Praktiken machen das Ganze skalierbar und betreibbar. Diese drei Themen sind nicht primär IT-Inhalte – sie sind *Steuerungsarchitektur*.

L1 – Wissen (Reproduktion und Einordnung)

- BPMS-Kernbausteine kennen: Prozessengine, Aufgaben-/Worklist, Forms, Regelwerk, Monitoring, Audit Trail, Versionierung.
- API-Verträge benennen: Endpoint, Input/Output, Fehlercodes, Auth, Rate Limits, Versionierung, Idempotenz.
- Integrationsmuster unterscheiden: synchroner API-Call vs. asynchrones Event/Message.
- Cloud-native Prinzipien einordnen: stateless Services, horizontale Skalierung, Observability (Logs, Metrics, Traces), Resilienz-Patterns.

L2 – Anwendung (Transfer auf konkrete Situationen)

- Einen BPMS-Blueprint für einen E2E-Prozess (Rollen, Tasks, SLAs, Eskalation, Audit Trail) entwerfen.
- Einen API-Vertrag mit Input/Output, Fehlerfällen und Versionierung formulieren.
- Sync- vs. Async-Entscheidung mit Speed/Robustheit-Trade-off begründen.
- Cloud-native Betriebsanforderungen ableiten (Skalierung, Observability, Resilienz, Security).

L3 – Management (Entscheidung und Verantwortung)

- “System of Record” pro Datenobjekt definieren und Data Ownership klären.
- Architekturpfade (event-driven vs. überwiegend sync) bewusst entscheiden – mit Begründung und Frühwarnsignal.
- Plattform als *Steuerungssystem für Entscheidungen* verstehen, nicht nur als technische Infrastruktur.
- “Minimum Viable Integration” priorisieren: lieber zwei kritische Integrationen stabil als zehn halbgar.

Roter Faden

Ein Prozess, der in einer BPMS-Engine läuft, ist nur so gut wie seine Integrationen. Eine API, die keinen klaren Vertrag hat, ist eine Wette. Eine Cloud-native Architektur ohne Observability ist Blindflug. Senior-Praxis verbindet alle drei: *Engine plus Verträge plus Beobachtbarkeit = E2E-Steuerungsarchitektur*.

1. Einstieg: Vom Diagramm zur integrierten Ausführung

In den vergangenen Tagen wurde Prozessmodellierung zunehmend technisch – vom Diagramm über das Repository zur Execution-Schicht. Heute schließt sich der Kreis: Prozesse laufen in *Systemen*, sie sprechen über *APIs*, und sie skalieren in *Cloud-nativen* Umgebungen. Das Verständnis dieser drei Bausteine ist die Voraussetzung dafür, BPM nicht als isolierte Disziplin zu verantworten, sondern als integriertes Steuerungssystem. (Dumas et al., 2018, Kap. 9; Freund & Rücker, 2019)

1.1 Was “E2E-Integration” bedeutet

End-to-End-Integration ist nicht “alle Systeme miteinander verbinden”. Es ist eine *durchgängige Sicht* entlang eines Prozesses: *Prozess + Daten + Systeme + Rollen + KPIs + Controls – als ein zusammenhängendes Bild*. Wenn nur eines dieser Elemente fehlt, fehlt die Steuerungsfähigkeit:

- Prozess ohne Daten – man modelliert ein Idealbild ohne Realität.
- Daten ohne Systeme – man hat Datenmodelle, aber niemand weiß, wo sie leben.
- Systeme ohne Rollen – niemand verantwortet die Schnittstelle.
- Rollen ohne KPIs – niemand misst, ob es funktioniert.
- KPIs ohne Controls – man sieht Abweichungen, ohne sie behandeln zu können.

1.2 Der “Model-Run-Improve”-Zyklus

Eine pragmatische Sicht auf den BPM-Lebenszyklus: (Dumas et al., 2018; Reinkemeyer, 2020)

1. **Model.** Soll-Prozess als BPMN oder VSM dokumentieren.
2. **Run.** Prozess in der Workflow-Engine ausführen, in echten Systemen.
3. **Measure.** KPIs erfassen, Process-Mining für Varianten und Engpässe.
4. **Improve.** Maßnahmen ableiten, Governance/Change anstoßen.

Wir bewegen uns heute auf der *Run*- und *Measure*-Schicht und schaffen die Voraussetzungen, dass *Improve* überhaupt möglich wird.

Eselsbrücke: B-A-C

Drei Bausteine der E2E-Steuerungsarchitektur:

BPMS – Modellieren, Ausführen, Überwachen, Optimieren.

APIs – Verträge zwischen Systemen: Input, Output, Fehler, Auth, Versionierung.

Cloud-native – Skalieren und beobachten in Containern, mit SLOs.

Wer B-A-C zusammen denkt, baut Steuerungsarchitektur. Wer einzeln denkt, baut Inseln.

2. BPMS – Business Process Management Systeme

Ein **BPMS** ist eine Software-Plattform, die vier Disziplinen vereint: *Modellieren, Ausführen, Überwachen, Optimieren* – der sogenannte **Closed Loop** der Prozesssteuerung. Bekannte Vertreter: Camunda Platform, Flowable, jBPM, IBM Business Automation Workflow, Pega; je nach Schwerpunkt mehr “Pure-BPM” oder “Low-Code-Suite”. (Freund & Rücker, 2019; Dumas et al., 2018, Kap. 9)

2.1 Kernbausteine eines BPMS

- **Prozessengine.** Führt BPMN-Modelle aus, verwaltet Zustände (Token), behandelt Events, ruft Tasks auf.
- **Aufgaben-/Worklist.** Liste der offenen User Tasks pro Benutzer/Rolle, mit Filtern und Priorisierung.
- **Forms.** Eingabe-Oberflächen für User Tasks (vom einfachen Web-Formular bis zur eingebetteten Custom-UI).
- **Regelwerk (DMN).** Decision Tables für Entscheidungslogik, die explizit aus dem Prozessmodell ausgelagert wird.
- **Monitoring/Dashboards.** Sicht auf laufende und abgeschlossene Cases, KPIs, Anomalien.
- **Audit Trail.** Unveränderliches Log aller Schritte, Entscheidungen und Datenänderungen – Pflicht in regulierten Umfeldern.
- **Versionierung.** Mehrere Prozess-Versionen können parallel existieren; Migration laufender Cases ist eine eigene Disziplin.
- **Case-/Human-Workflow.** Ergänzung zur strengen BPMN-Steuerung für wissensintensive Fälle (CMMN-Logik).

2.2 Nutzen und Risiken eines BPMS

	Nutzen	Risiken
Standardisierung	gleiche Abläufe, vergleichbare Daten	“Schablone für alles”
Durchlaufzeit	weniger Liegezeit, klare Eskalation	Overengineering, Sonderlocken
Compliance	Audit Trail, klare Verantwortung	Falsche Prozessgrenzen
Transparenz	Dashboards, Variantenanalyse	“False Compliance” (Papier ok, Realität nicht)

2.3 BPMS-Blueprint für Customer-Complaint-Handling

Wir nehmen einen realistischen Mini-Prozess: *Customer Complaint Handling*. Reklamation kommt rein → klassifizieren → ggf. Rückfrage → Entscheidung Kulanz/-Retoure → Abwicklung → Abschluss → Reporting. SLAs: 48 h Erstreaktion, 10 Tage Abschluss. Daten in CRM, ERP, E-Mail.

Blueprint-Skizze (Kästchenmodell)

- **Trigger:** Message Event “Complaint empfangen” (E-Mail-Eingang, Webformular, oder API).

- **User Task “Klassifizieren”** (Lane Customer Service) – 30 min SLA. Daten: Typ, Schweregrad, Produktbezug.
- **XOR Gateway “Klärung nötig?”**
Ja → User Task “Rückfrage stellen” + Receive Task “Antwort erhalten” (Timer 5 Tage).
Nein → direkt zu Entscheidung.
- **Business Rule Task “Kulanz vs. Retoure”** (DMN-Tabelle) – Schwellwert, Kundenwert, Produkthistorie.
- **User Task “Abwicklung”** (Lane Sales Ops oder Logistik) – abhängig vom Ergebnis.
- **Service Task “Kundenkommunikation”** (Mail/Push) + Audit-Logging.
- **End Event “Abgeschlossen”** – Reporting-Trigger.

SLAs und Eskalation

- Erstreaktion 2 h (Klassifizierung muss innerhalb 2 h erfolgen, sonst Eskalation an Schichtleitung).
- Lösung 10 Tage (gesamter Case in 10 Werktagen abgeschlossen).
- Eskalation nach 4 h ohne Aktivität an Duty Manager (Timer Boundary Event).

KPIs

- **SLA-Einhaltung** – Anteil der Cases mit Erstreaktion innerhalb SLA. Owner: Head of Customer Service.
- **Rework-Quote** – Anteil der Cases mit Wiedereröffnung nach Abschluss. Owner: Process Owner.
- **Kundenzufriedenheit** – NPS oder CSAT pro abgeschlossenem Case. Owner: VP Customer Experience.

MVP-Entscheidung

Eine **Funktion gehört rein**: der Audit Trail. Ohne Audit Trail kein BPMS-Wert.
Eine **Funktion bleibt vorerst draußen**: die automatische Kategorisierung per ML. Sie ist verlockend, aber sie erfordert Trainingsdaten, die zum Start nicht in der nötigen Qualität existieren. Stattdessen: regelbasierte Vorklassifikation in der DMN-Tabelle. Trigger für die Erweiterung: >3000 Cases im Trainingskorpus mit Label.

BPMS ist nicht Workflow-Engine ist nicht Modellierungs-Tool

Ein BPMS umschließt eine Workflow-Engine, ein Modellierungs-Tool, ein Forms-System, ein Regelwerk, Monitoring und Audit. Wer mit “wir kaufen eine Workflow-Engine” startet und denkt, das sei BPMS, unterschätzt den organisatorischen Anteil – Forms, Rollen, Audit, Reporting machen 70 % der Komplexität aus.

3. APIs – Verträge zwischen Systemen

Eine **API** (Application Programming Interface) ist ein *Vertrag* zwischen zwei Systemen. Die wichtigsten Vertragsbestandteile: Endpoint (Adresse), Input (was

wird mitgegeben), Output (was kommt zurück), Fehlercodes (was geht schief), Authentifizierung (wer darf), Rate Limits (wie oft), Versionierung (welche Ausgabe). (Fielding, 2000; Richardson, 2018; Newman, 2021)

3.1 Vertragsbestandteile im Detail

- **Endpoint.** URL/Adresse, unter der die Funktion erreichbar ist. Z. B. POST /v2/orders/{orderId}
- **Input.** Schema der erwarteten Felder (Pflicht, optional, Typ, Wertebereich).
- **Output.** Schema der Antwort (Erfolg, Fehler, Status, Body).
- **Fehlercodes.** HTTP-Status plus optional fachliche Fehlercodes (z. B. ERR_ORDER_NOT_FOUND).
- **Auth.** OAuth-2.0, API-Key, mTLS – wer ist berechtigt?
- **Rate Limits.** Anzahl Aufrufe pro Zeitfenster, um Last zu steuern.
- **Versionierung.** URL-Path, Header oder Content-Type-Negotiation; Lebenszyklen mit Deprecation-Zeitfenstern.
- **Idempotenz.** Mehrfacher Aufruf mit gleicher ID liefert dasselbe Ergebnis – kritisch für Retries.

3.2 Sync vs. Async – der zentrale Designentscheid

Synchrone API-Aufrufe

- Aufrufer wartet auf Antwort. Klassisches REST über HTTPS.
- Vorteil: einfach, sofortige Rückmeldung, Fehler direkt sichtbar.
- Nachteil: koppelt Verfügbarkeit (wenn Aufgerufener down, blockiert Aufrufer), erschwert horizontale Skalierung.
- Geeignet für: Abfragen, kurze Operationen, Benutzerinteraktionen.

Asynchrone Events/Messages

- Sender legt Event/Message in Kanal (Kafka, RabbitMQ, Cloud-Bus), Empfänger holt asynchron.
- Vorteil: Entkopplung, lastrobust, einfache Skalierung.
- Nachteil: Komplexere Fehlerbehandlung, “eventual consistency”, Diagnose schwieriger.
- Geeignet für: Hintergrundverarbeitung, Skalierungs-Spitzen, Domänen-Events (“Order placed”).

3.3 API-Vertrags-Template

Ein praktikables Template für jede neue API:

Name/Zweck	POST /v2/shipments/{id}/status – Statusabfrage Sendung
Input	Path: shipmentId; Header: X-Correlation-ID, Authorization
Output	200: {status, estimatedDelivery, carrier, lastUpdate}; 404: not found
Fehlerfälle	401 Unauthorized; 404 Not Found; 503 Service Unavailable
Security	OAuth-2.0 Bearer Token; Scope “shipment:read”
Versionierung	URL-Path /v2/; /v1/ deprecated bis 30.06.2026
Monitoring	Log mit X-Correlation-ID; SLO: 95 % < 200 ms

3.4 Zwei Beispiel-APIs für Customer-Complaint-Integration

API 1 – GetOrderStatus (sync)

- Input: orderId (Pflicht), customerId (optional).
- Output: status, currentStep, lastUpdate, items[].
- Fehlerfälle: 401 (Auth fail), 404 (Order nicht gefunden), 503 (Backend nicht erreichbar).
- Security: OAuth-2.0 mit Scope “order:read”.
- Versionierung: /v2/orders/{id}.
- Monitoring: Correlation-ID, SLO 95 % < 300 ms.

API 2 – CreateReturn (sync, mit Idempotenz)

- Input: orderId, reasonCode, items[], idempotencyKey.
- Output: returnId, status, instructions.
- Fehlerfälle: 401, 409 (Conflict, idempotencyKey bereits genutzt), 422 (validation), 503.
- Security: OAuth-2.0 mit Scope “return:write”.
- Idempotenz: idempotencyKey verhindert doppelte Returns bei Retry.

3.5 Fehler-Strategien im Workflow

Fehlerfall	Workflow-Reaktion
Timeout	Retry max. 3× mit Exponential Backoff, dann Manual Fallback.
Daten fehlen (4xx)	User Task “Daten ergänzen” zurück an Sender.
Auth-Fehler (401)	Eskalation an System-Owner, kein Retry.
Service unavailable (5xx)	Cache/Fallback nutzen, asynchron erneut versuchen.

API ist ein Vertrag, kein Nebenprodukt

Eine API ohne explizit dokumentierten Vertrag ist eine Wette, dass die andere Seite immer dasselbe meint. Sobald die andere Seite ein Update macht, ohne den Vertrag zu pflegen, wandert die Wette ins Risiko. *API-Verträge gehören in das Repository, sind versioniert, haben Owner.*

3.6 Sync vs. Async – Faustregel

- **Sync**, wenn: Benutzer wartet auf Antwort, kurzer Aufruf, niedrige Last, einfache Fehlerbehandlung gewünscht.
- **Async**, wenn: lange laufende Verarbeitung, Lastspitzen, mehrere Empfänger desselben Events (Fan-out), oder Entkopplung gewünscht.

4. Cloud-native Skalierung und Betrieb

Cloud-native ist nicht “in der Cloud betrieben”. Es ist ein Architekturansatz, der bewusst die Eigenschaften der Cloud nutzt: horizontale Skalierung, automati-

sches Deployment, Resilienz durch Replikation, beobachtbarer Betrieb. (Newman, 2021; Burns et al., 2022)

4.1 Sechs Prinzipien

1. **Stateless Services.** Dienste halten keinen Zustand lokal – der Zustand liegt in Datenbanken oder Message-Bussen. Voraussetzung für horizontale Skalierung.
2. **Horizontale Skalierung.** Mehr Instanzen statt größere Maschinen. Wirkt nur, wenn Services stateless sind.
3. **Container und Orchestrierung.** Container (Docker) als Verpackung, Kubernetes als Orchestrator. Wiederholbare Deployments.
4. **Observability.** Drei Säulen: *Logs* (was ist passiert), *Metrics* (wie viel), *Traces* (wie hängt es zusammen über Service-Grenzen).
5. **Resilienz.** Circuit Breaker, Retries, Timeouts, Bulkheads – Muster, die Fehler isolieren, statt sie zu propagieren.
6. **Deklarative Infrastruktur.** Konfiguration als Code (Infrastructure-as-Code), Pipelines (CI/CD) statt manueller Deployments.

4.2 SLAs und SLOs

- **SLA** (Service Level Agreement) – vertraglich zugesicherte Service-Qualität (z. B. 99,9 % Verfügbarkeit pro Monat).
- **SLO** (Service Level Objective) – internes Ziel, das hinter dem SLA mit Puffer liegt (z. B. 99,95 %). Wenn SLO verletzt wird, gibt es Maßnahmen *bevor* der SLA reißt.
- **SLI** (Service Level Indicator) – die Messgröße selbst (Antwortzeit, Verfügbarkeit, Fehlerrate).

4.3 Resilienz-Patterns

- **Circuit Breaker.** Nach mehreren Fehlern wird der Aufruf temporär blockiert – der Service hat Zeit, sich zu erholen, und der Aufrufer wartet nicht endlos.
- **Retry mit Exponential Backoff.** Wiederholte Versuche mit immer längeren Pausen, statt Hammer-Retries.
- **Timeout.** Jeder Aufruf hat eine Obergrenze – ohne Timeout kann ein einzelner Hänger die ganze Kette blockieren.
- **Bulkhead.** Trennung von Ressourcen-Pools, damit ein überlasteter Service nicht andere mitreißt.
- **Idempotenz** (auf Aufrufer-Seite). Erlaubt sicheres Wiederholen, wenn Unsicherheit über vorigen Aufruf besteht.

4.4 Security – Zero-Trust-Grundgedanke

Zero Trust bedeutet: kein internes Netzwerk wird “vertraut”. Jede Anfrage muss sich authentifizieren und autorisieren, auch innerhalb der Organisation. Konsequenzen für Architektur: mTLS zwischen Services, Token-basierte Auth, Secrets aus Vault statt Konfig-Datei.

4.5 Deployment-Strategien (konzeptionell)

- **Blue/Green.** Zwei Umgebungen, Wechsel per Switch. Schneller Rollback, doppelter Ressourcenbedarf.
- **Canary.** Neue Version auf wenige Nutzer/Prozente, schrittweise erhöhen. Frühe Erkennung von Problemen.
- **Rolling.** Schrittweise Ablösung der alten Instanzen. Geringerer Ressourcenbedarf, etwas längere Migration.
- **Feature Toggle.** Funktion deployed, aber per Schalter aus/ein – erlaubt Test in Prod ohne Risiko.

Eselsbrücke: L-M-T

Die drei Säulen der Observability:

Logs – was ist passiert, in welcher Reihenfolge?

Metrics – wie viel, wie schnell, wie oft?

Traces – wie hängt es service-übergreifend zusammen?

Cloud-native ohne L-M-T ist Blindflug. Wer im Fehlerfall raten muss, hat die Plattform nicht im Griff.

5. Fallstudie: EuroLogix – E2E Shipment Exception Handling

EuroLogix ist ein Logistikdienstleister. Der E2E-Prozess “Shipment Exception Handling” ist langsam, hat viele Medienbrüche, erzeugt Kundenbeschwerden. Systeme: TMS (Transport-Management), CRM, E-Mail, Data Warehouse. Volumen schwankt stark (Peaks zu Saisons).

5.1 Ausgangslage und Restriktionen

- Zeit: 9 Wochen für Release 1.
- Budget: 150 000 €.
- Legacy-TMS mit eingeschränkten APIs.
- Security-Anforderungen erhöht (regulatorischer Druck).
- Peak-Lastdaten lückenhaft.

5.2 Schritt 1 – BPMS-gestützter Prozess

Start (Event “ExceptionRaised” aus TMS, asynchron)

□ **User Task** “Klassifizieren” (Lane Exception Desk; SLA 30 min).

□ **XOR “Kritisch?”**

Ja: □ **Service Task** “Sofortige Eskalation” + **Service Task** “Kundeninfo” (proaktiv).

Nein: □ **User Task** “Ursache klären” (CRM/TMS-Lookup, SLA 4 h).

□ **User/Service Task** “Maßnahme” (Umroute, Neuversand, Gutschrift).

□ **Service Task** “Kundenkommunikation” (Mail/Push) + Audit-Logging.

□ **Ende “Geschlossen”** + Reporting.

SLAs und Eskalation

- Erstreaktion: 2 h ab Event.

- Lösung: 24 h ab Klassifizierung als nicht-kritisch.
- Eskalation: nach 4 h ohne Bewegung an Duty Manager (Timer Boundary).
- Notfall-Eskalation: bei “kritisch” sofort an Duty Manager, parallel.

5.3 Schritt 2 – Drei Integrationspunkte

Event ExceptionRaised (async)

- Quelle: TMS via Message Broker.
- Felder: caseId, shipmentId, timestamp, severity, sourceSystem.
- Idempotenz: caseId als Schlüssel.

API GetShipmentStatus (sync)

- Endpoint: GET /v2/shipments/{id}.
- Auth: OAuth-2.0, Scope “shipment:read”.
- Fehler-Fallback bei 503: Cache letzter bekannter Status nutzen, Hinweis im UI “Daten möglicherweise veraltet”.
- Fehler-Fallback bei Timeout: nach 2 s, Retry 2×, dann manuelle Prüfung.

API/Event NotifyCustomer

- Bei kleineren Mitteilungen: Event in Mail-Bus.
- Bei rechtlich relevanten Mitteilungen: synchroner API-Call ans CRM mit Bestätigung.
- Audit-Logging in beiden Fällen Pflicht (mit korrelierter caseId).

5.4 Schritt 3 – Cloud-native Anforderungen

- **Auto-Scaling** bei Peak-Saisons (Black Friday, Weihnachten); horizontale Skalierung der Engine-Worker.
- **Observability**: zentrale Logs (mit caseId-Korrelation), Metrics (Cases/h, SLA-Hit-Rate, p95-Latenz), Traces über Service-Grenzen.
- **Resilienz**: Circuit Breaker an TMS-API, Retry-Policy mit Exponential Backoff, Idempotenz via caseId.
- **Security**: Secrets in zentralem Vault, mTLS zwischen Services, Audit-Trail unveränderlich.
- **Deployment**: Blue/Green für die BPMS-Engine, Canary für Service Tasks mit hohem Änderungsrisiko.

5.5 Schritt 4 – E2E-Methodenintegration

Das Methodenkonzept verbindet die Bausteine:

- BPMN als Soll-Modell, gepflegt im Repository.
- VSM für Engpässe (Klassifizieren oft Engpass, weil Klassifikations-Skills knapp).
- KPIs als Steuerung (SLA-Hit-Rate, Eskalationsquote, NPS).
- Process Mining für Varianten (welche Sonderfälle treten häufig auf?).
- Monatliches **Process Review Board** mit Process Owner, IT-Lead, Operations.

5.6 Schritt 5 – MVP-Entscheidung

Drin in Release 1:

- BPMS-Case-Handling für Standardfälle (80 % Coverage).
- Event ExceptionRaised vom TMS.
- Status-API GetShipmentStatus (mit Cache-Fallback).
- SLA-Eskalation und Audit Trail.
- Dashboard mit Top-5 KPIs.

Bewusst nicht in Release 1:

- Full Data Warehouse Rebuild (zu großer Umfang).
- ML-basierte Klassifikation (zu wenig Trainingsdaten).
- Mobile-App für Exception Desk (kann Release 2 werden).

5.7 Frühwarnsignale

- SLA-Hit-Rate sinkt unter 90 % in einer Woche → Eskalation an Process Owner.
- Eskalationsquote steigt über 20 % → Slice oder Ressourcen prüfen.
- TMS-API-Verfügbarkeit unter 99 % pro Tag → Architektur-Review.

6. SOA, ESB, API-Ökonomie – ein kurzer historischer Bogen

Wer heute APIs entwirft, profitiert von einem historischen Kontext, der die letzten 25 Jahre prägt. (Erl, 2005; Richardson, 2018)

6.1 Service-orientierte Architektur (SOA)

In den 2000ern setzte sich das Konzept der **SOA** durch: fachlich geschnittene Services mit klaren Verträgen, oft koordiniert über einen **Enterprise Service Bus (ESB)**. Ziel: Wiederverwendung, Standardisierung, Entkopplung. In der Praxis kollabierte SOA oft an zentralen ESBs, die zu Single Points of Failure wurden.

6.2 Microservices

Die Microservices-Bewegung der 2010er antwortete mit kleineren, eigenständigen Services und der Vermeidung zentraler Koordinationsknoten. Stattdessen: *Smart Endpoints, dumb pipes*. Jede Integration explizit, keine versteckten Transformationen im Bus.

6.3 Cloud-native API-Ökonomie

Heute leben wir in einer Welt, in der APIs als Produkt verstanden werden – mit Produktmanagement, Versionierung, Marketing-ähnlichen Adoption-Strategien. API-Gateways übernehmen Cross-Cutting Concerns (Auth, Rate Limiting, Logging), während die fachlichen Services darunter laufen.

“Smart Endpoints, dumb pipes” – ein Senior-Prinzip

Wer Integrationen baut, sollte Logik in die Services legen, nicht in die Mittelschicht. Bus oder Gateway lösen Cross-Cutting-Themen (Auth, Rate Limit, Logging); fachliche Logik gehört dort hin, wo die Daten sind. Sonst entsteht ein neuer ESB-Monolith mit anderem Namen.

7. Management-Perspektive: Plattform als Steuerungssystem

Auf der Wissens-Ebene sind BPMS, APIs und Cloud-native technische Themen. Auf Anwendungs-Ebene sind sie Handwerk. Auf Management-Ebene sind sie *Plattform-Entscheidungen* – mit fünf- bis sechststelligen Konsequenzen über Jahre. (Humble & Molesky, 2011; vom Brocke & Mendling, 2021)

7.1 Zwei zentrale Zielkonflikte

1. **Architekturpfad: event-driven vs. überwiegend sync.** Event-driven bringt Skalierbarkeit und Entkopplung – und Debugging-Komplexität. Überwiegend sync ist verständlicher – und koppelt Verfügbarkeit. Empfehlung: *sync für Benutzer-Interaktion, async für Hintergrund-Workflows und Domänen-Events*; explizite Architekturentscheidung pro Use Case, nicht “one size fits all”.
2. **Standardisierung vs. Team-Autonomie.** Zentrale API-Standards (REST-Konventionen, Versionierung, Auth) erleichtern Wiederverwendung – und nehmen Teams Geschwindigkeit. Pure Team-Autonomie erzeugt API-Wildwuchs. Praktikabel: *verbindliche Standards für Cross-Cutting-Themen, lokale Freiheit bei fachlicher Modellierung*.

7.2 Operating Model – wer verantwortet was

- **Platform Engineering Team.** Betreibt BPMS-Plattform, API-Gateway, Observability-Stack – als interner Service-Provider.
- **Domain Teams.** Entwerfen und betreiben fachliche Services und Workflows; nutzen die Plattform.
- **API-Owner pro Domäne.** Verantwortet API-Verträge, Versionierung, SLAs.
- **Process Owner.** Fachlich verantwortlich für E2E-Prozess; entscheidet über Workflow-Logik.
- **Security/Compliance.** Definiert Standards, prüft Audit Trail.

7.3 System of Record – Datenverantwortung

Pro Datenobjekt muss ein **System of Record (SoR)** festgelegt sein – das System, das die “Wahrheit” hält. Beispielsweise:

- Kunde: CRM.
- Auftrag: ERP.
- Sendung: TMS.
- Reklamation: BPMS-Case.

Andere Systeme dürfen Kopien halten, müssen aber bei Diskrepanz das SoR konsultieren. Diese Vereinbarung schriftlich, mit Data Owner pro SoR.

7.4 Entscheidungen unter Unsicherheit

- **MVP-Scope.** Welche zwei Integrationen zuerst – mit “Stop Doing”-Liste der bewusst zurückgestellten. Frühwarnsignal: Wenn nach Release 1 nicht mindestens eine KPI um 20 % bewegt wurde, war die Auswahl falsch.
- **Plattform-Wahl.** BPMS Camunda vs. Flowable vs. Pega – Entscheidung gilt 3 bis 5 Jahre. Frühwarnsignal: zwei kritische Use Cases scheitern an Plattform-

Limits → Re-Evaluation.

- **Team-Topologie.** Platform Team zentral vs. in jedem Domain Team eingebettet. Frühwarnsignal: zentrale Wartezeit auf Plattform-Anfragen > 2 Wochen → Self-Service-Modell ausbauen.

7.5 Risikoarchitektur

1. **Daten-Risiko.** Falsche oder fehlende Daten in Workflows. Mitigation: Pflichtfelder, Schema-Validierung, Data Quality SLAs.
2. **Integrations-Risiko.** APIs ändern sich, ohne dass es jemand merkt. Mitigation: Versionierung mit Deprecation-Fenster, Contract Tests, API-Change-Notifications.
3. **Verfügbarkeits-Risiko.** Ein kritischer Service down, kompletter Prozess steht. Mitigation: Circuit Breaker, Fallbacks, Degraded-Mode definieren.
4. **Security-Risiko.** Token-Diebstahl, Secrets-Leak. Mitigation: Vault, mTLS, regelmäßige Rotation, Audit auf Anomalien.
5. **Compliance-Risiko.** Audit Trail unvollständig oder manipuliert. Mitigation: unveränderlicher Log-Speicher, separate Lese- und Schreibrechte.

7.6 “Minimum Viable Integration”

Die häufigste Fehleinschätzung in Plattform-Programmen ist Überdimensionierung am Start. Senior-Praxis verfolgt das Gegenteil: *lieber zwei Integrationen stabil als zehn halbgar*.

- Wähle zwei Integrationen mit höchstem Wert und niedrigstem Risiko.
- Baue sie vollständig: Vertrag, Tests, Monitoring, Audit.
- Lerne – und nutze die Erkenntnisse für das nächste Set.
- Niemals “alle gleichzeitig” – die Komplexität wächst exponentiell.

Plattform ist Steuerungssystem, nicht Infrastruktur

Eine Plattform ohne Operating Model ist eine Sammlung von Werkzeugen. Erst durch Owner, Verträge, SLOs, Audit und KPIs wird sie zum Steuerungssystem. Wer in Plattform-Themen rein technisch denkt, baut die richtige Maschine – ohne dass jemand sie verantwortet.

8. Take-aways aus Tag 6

1. **BPMS = Modellieren + Ausführen + Überwachen + Optimieren.** Closed Loop – alle vier Schritte gehören zusammen.
2. **API = Vertrag.** Endpoint, Input, Output, Fehler, Auth, Versionierung, Idempotenz – vollständig oder gar nicht.
3. **Sync für Interaktion, Async für Hintergrund.** Entscheidung pro Use Case, nicht pauschal.
4. **Cloud-native = Skalierung + Beobachtbarkeit + Resilienz.** L-M-T (Logs, Metrics, Traces) ist Pflicht.
5. **System of Record pro Datenobjekt.** Eine Wahrheit, klar benannte Data Owner.

6. **“Minimum Viable Integration”**. Lieber zwei stabil als zehn halbgar.
7. **Smart Endpoints, dumb pipes**. Fachliche Logik in Services, nicht in der Mittelschicht.
8. **Plattform braucht Operating Model**. Owner, SLOs, Audit, KPIs – sonst bleibt es Infrastruktur ohne Steuerung.

9. Literatur

Im Skript verwendete und für die Vertiefung empfohlene Quellen. Zitierstil: APA-7.

Burns, B., Villalba, E., Strebel, D., & Evenson, L. (2022).

Kubernetes: Up and running – Dive into the future of infrastructure (3rd ed.). O'Reilly.

Dumas, M., La Rosa, M., Mendling, J., & Reijers, H. A. (2018).

Fundamentals of business process management (2nd ed.). Springer. <https://doi.org/10.1007/978-3-662-56509-4>

Erl, T. (2005).

Service-oriented architecture: Concepts, technology, and design. Prentice Hall.

Fielding, R. T. (2000).

Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine]. <https://www.ics.uci.edu/~fielding/pu>

Freund, J., & Rücker, B. (2019).

Real-life BPMN: Includes an introduction to DMN (4th ed.). CreateSpace.

Humble, J., & Molesky, J. (2011).

Why enterprises must adopt devops to enable continuous delivery. *Cutter IT Journal*, 24(8), 6–12.

Newman, S. (2021).

Building microservices: Designing fine-grained systems (2nd ed.). O'Reilly.

Reinkemeyer, L. (Hrsg.). (2020).

Process mining in action: Principles, use cases and outlook. Springer. <https://doi.org/10.1007/978-3-030-40172-6>

Richardson, C. (2018).

Microservices patterns: With examples in Java. Manning.

vom Brocke, J., & Mendling, J. (Hrsg.). (2021).

Business process management cases: Digital innovation and business transformation in practice (Vol. 2). Springer. <https://doi.org/10.1007/978-3-662-63047-1>

10. Glossar

Die wichtigsten Begriffe des Tages – kurz definiert, in alphabetischer Reihenfolge.

API

Application Programming Interface. Vertrag zwischen zwei Systemen mit Endpoint, Input, Output, Fehlern und Auth.

API-Gateway

Vorgeschalteter Dienst für Cross-Cutting-Themen (Auth, Rate Limit, Logging) bei API-Aufrufen.

Audit Trail

Unveränderliches Log aller Schritte, Entscheidungen und Datenänderungen eines Workflow-Laufs.

BPMS

Business Process Management System. Plattform für Modellieren, Ausführen, Überwachen, Optimieren von Prozessen.

Blue/Green Deployment

Strategie mit zwei Umgebungen, Wechsel per Switch.

Canary Deployment

Strategie, bei der eine neue Version zuerst auf wenige Nutzer oder Anteile aktiviert wird.

Cloud-native

Architekturansatz mit stateless Services, Containern, horizontaler Skalierung, Observability und Resilienz.

Circuit Breaker

Resilienz-Pattern: nach mehreren Fehlern wird ein Aufruf temporär blockiert.

CMMN

Case Management Model and Notation. OMG-Standard für wissensintensive Fälle, ergänzend zu BPMN.

Container

Verpackung einer Anwendung samt Abhängigkeiten (z.B. Docker), Voraussetzung für Cloud-native Deployments.

DMN

Decision Model and Notation. OMG-Standard für Entscheidungsregeln, in BPMS oft als Business Rule Task aufgerufen.

ESB

Enterprise Service Bus. Zentrale Integrationsschicht; oft kritisch wegen Single-Point-of-Failure-Risiko.

Event

Asynchrone Mitteilung, dass etwas geschehen ist ("OrderPlaced"); wird in Message Broker oder Bus gelegt.

Idempotenz

Eigenschaft eines API-Aufrufs: wiederholter Aufruf mit gleicher ID liefert dassel-

be Ergebnis.

Kubernetes

Verbreiteter Container-Orchestrator für Cloud-native Anwendungen.

Microservices

Architekturstil mit kleinen, eigenständigen Services und expliziten Integrationen.

Observability

Fähigkeit, das Verhalten eines Systems von außen zu verstehen; drei Säulen: Logs, Metrics, Traces.

Process Engine

Kernkomponente eines BPMS, die BPMN-Modelle ausführt und Zustände verwaltet.

REST

Architekturstil für APIs auf Basis von HTTP-Ressourcen, ursprünglich beschrieben von Fielding (2000).

Rate Limit

Obergrenze für Aufrufe pro Zeitfenster; Schutz gegen Überlast und Missbrauch.

Resilienz-Pattern

Sammelbegriff für Muster wie Circuit Breaker, Retry, Timeout, Bulkhead, Idempotenz.

SLA

Service Level Agreement. Vertraglich zugesicherte Servicequalität.

SLO

Service Level Objective. Internes Ziel hinter dem SLA, oft mit Puffer.

SoR

System of Record. Das System, das pro Datenobjekt die "Wahrheit" hält.

Zero Trust

Sicherheitsansatz, der internen Netzwerken nicht "vertraut" und jede Anfrage authentifiziert/autorisiert.